



UNIVERSITY OF PETROLEUM AND ENERGY
STUDIES
DEHRADUN

Modeling and Stimulation

Lab Experiment 7

MTECH-COMPUTER SCIENCE
ENGINEERING
CYBER SECURITY AND FORENSICS

Name: Jigesh Sheoran

SAP ID: 590025428

Aim

To implement parallel matrix multiplication using multiprocessing, and analyze performance metrics such as speedup and scalability.

Theory :

Parallel and Distributed Simulation involves dividing a computational task into smaller sub-tasks that can be executed simultaneously across multiple processors.

Matrix multiplication is a computationally intensive operation with time complexity $O(n^3)$. By partitioning the matrix into smaller chunks and assigning them to different processors, execution time can be reduced.

Parallel Concept:

- Divide matrix rows among processes
- Each process computes partial results
- Combine results to get final matrix

Performance Metrics:

- **Execution Time (T)**
- **Speedup (S):**

$$S = \{T_{\text{serial}}\} / \{T_{\text{parallel}}\}$$

- **Efficiency (E):**

$$E = \{S\} / \{P\}$$

where P = number of processors

Methodology:

2.1 Data Partitioning

- Matrix A rows are divided among processes
- Matrix B is shared

2.2 Parallel Execution

- Each process computes multiplication for its assigned rows
- Synchronization handled using process join

2.3 Performance Evaluation

- Measure serial execution time
- Measure parallel execution time
- Compute speedup and efficiency

Algorithm:

1. Input matrices A and B
2. Divide rows of A among processes
3. Each process computes its portion
4. Combine results
5. Measure execution time
6. Calculate speedup and efficiency

Code:

```
lab7.py > parallel_multiply
1  # Author: Jigesh Sheoran
2  # Experiment: Parallel and Distributed Simulation (Matrix Multiplication)
3  # Last Modified : 19 / 03 / 2026
4
5  import multiprocessing as mp
6  import time
7  import random
8  import matplotlib.pyplot as plt
9
10 # Generate Matrix
11 def generate_matrix(n):
12     return [[random.randint(1, 10) for _ in range(n)] for _ in range(n)]
13
14 # Serial Multiplication
15 def serial_multiply(A, B):
16     n = len(A)
17     result = [[0]*n for _ in range(n)]
18
19     for i in range(n):
20         for j in range(n):
21             for k in range(n):
22                 result[i][j] += A[i][k] * B[k][j]
23
24     return result
25
26 # Worker Function (Parallel)
27 def multiply_row(args):
28     A, B, row = args
29     n = len(B)
30     result_row = []
31
32     for j in range(n):
33         val = 0
34         for k in range(n):
35             val += A[row][k] * B[k][j]
36         result_row.append(val)
37
38     return (row, result_row)
39
40 # Parallel Multiplication
41 def parallel_multiply(A, B, processes):
42     n = len(A)
43     pool = mp.Pool(processes)
44
45     tasks = [(A, B, i) for i in range(n)]
46     results = pool.map(multiply_row, tasks)
47
48     pool.close()
49     pool.join()
50
```

```

50
51     result = [None]*n
52     for row, value in results:
53         result[row] = value
54     return result
55
56 # MAIN
57 if __name__ == "__main__":
58
59     n = 200 # matrix size (adjust if slow)
60
61     A = generate_matrix(n)
62     B = generate_matrix(n)
63
64     print(f"\nMatrix Size: {n}x{n}")
65
66     # SERIAL EXECUTION
67     start = time.time()
68     serial_result = serial_multiply(A, B)
69     serial_time = time.time() - start
70
71     print(f"Serial Time: {serial_time:.4f} sec")
72
73     # PARALLEL EXECUTION (default cores)
74     start = time.time()
75     parallel_result = parallel_multiply(A, B, mp.cpu_count())
76     parallel_time = time.time() - start
77
78     print(f"Parallel Time: {parallel_time:.4f} sec")
79
80     speedup = serial_time / parallel_time
81     efficiency = speedup / mp.cpu_count()
82
83     print(f"Speedup: {speedup:.2f}")
84     print(f"Efficiency: {efficiency:.2f}")
85
86     # PERFORMANCE GRAPH
87     process_counts = [1, 2, 4] # adjust based on your CPU
88     speedups = []
89     efficiencies = []
90
91     for p in process_counts:
92         start = time.time()
93         parallel_multiply(A, B, p)
94         parallel_time = time.time() - start
95
96         sp = serial_time / parallel_time
97         eff = sp / p
98
99         speedups.append(sp)
100        efficiencies.append(eff)

```

```

101
102     print(f"Processes: {p}, Speedup: {sp:.2f}, Efficiency: {eff:.2f}")
103
104     # PLOT 1: Speedup
105     plt.figure()
106     plt.plot(process_counts, speedups, marker='o')
107     plt.title("Speedup vs Number of Processes")
108     plt.xlabel("Number of Processes")
109     plt.ylabel("Speedup")
110
111     # PLOT 2: Efficiency
112     plt.figure()
113     plt.plot(process_counts, efficiencies, marker='o')
114     plt.title("Efficiency vs Number of Processes")
115     plt.xlabel("Number of Processes")
116     plt.ylabel("Efficiency")
117
118     plt.show()

```

Terminal Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

/usr/local/bin/python3 "/Users/sheoraninfosec/Documents/Documents - Jigesh's Ma
● sheoraninfosec@Jigeshs-MacBook-Air Lab 7 % /usr/local/bin/python3 "/Users/sheor
b7.py"

Matrix Size: 200x200
Serial Time: 0.4207 sec
Parallel Time: 0.6823 sec
Speedup: 0.62
Efficiency: 0.08
Processes: 1, Speedup: 0.76, Efficiency: 0.76
Processes: 2, Speedup: 0.98, Efficiency: 0.49
Processes: 4, Speedup: 1.00, Efficiency: 0.25
○ sheoraninfosec@Jigeshs-MacBook-Air Lab 7 % █

```

Figure 1 :

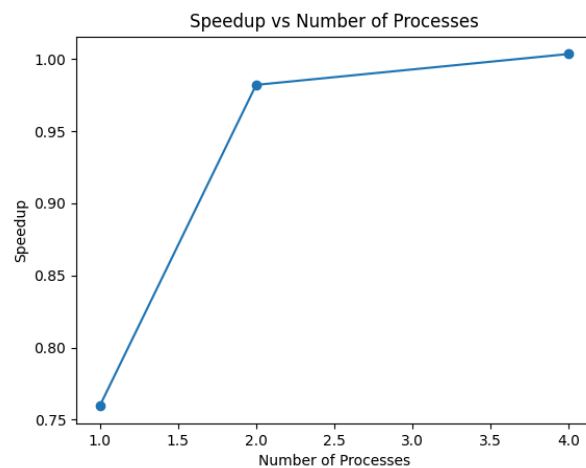
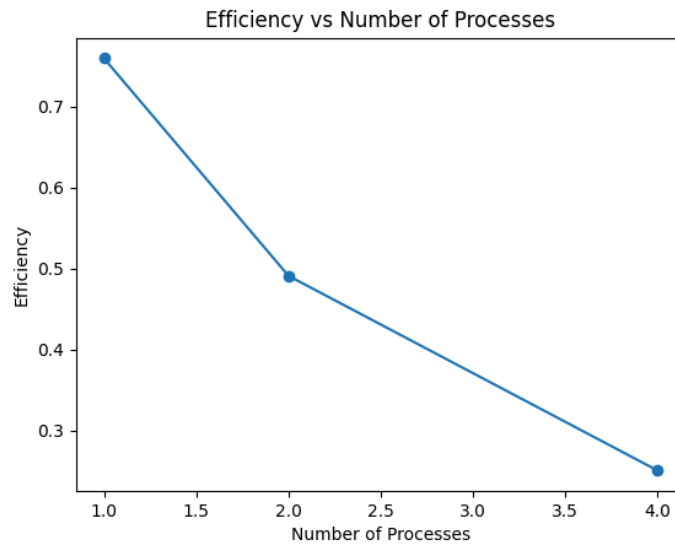


Figure 2 :



Results:

The parallel matrix multiplication algorithm was executed for a matrix of size 200×200 . The execution times for both serial and parallel implementations were measured.

From the obtained results:

- Serial execution time was **0.4207 seconds**
- Parallel execution time was **0.6823 seconds**
- The computed speedup was **0.62**
- Efficiency was **0.08**

It was observed that parallel execution did not outperform serial execution for this matrix size due to overhead involved in multiprocessing.

Observation Table:

Number of Processes	Speedup	Efficiency
1	0.76	0.76
2	0.98	0.49
4	1.00	0.25

Conclusion:

The experiment demonstrates that parallel computing does not always guarantee performance improvement. In this case, the parallel execution was slower than the serial execution due to overhead from process creation, communication, and synchronization.

The observed behavior aligns with Amdahl's Law, where the non-parallelizable portion limits overall speedup.

Final observations :

- For smaller problem sizes, parallelism introduces overhead that reduces performance
- Speedup increases slightly with more processes but remains limited
- Efficiency decreases as the number of processes increases
- Parallel computation becomes beneficial only for larger datasets

Performance overhead due to process creation and inter-process communication dominates for smaller matrix sizes.

-----END OF DOCUMENT -----